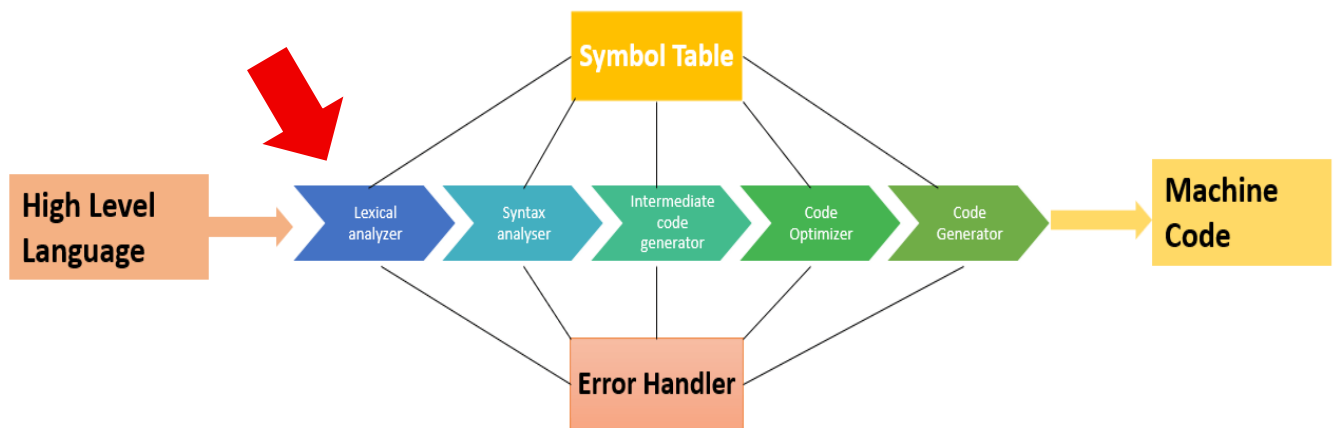




Rapport projet de compilation

GAL

Cohen Ayala | Compilation | 15/05/2026

Introduction :

Ce projet a pour objectif de créer un générateur d'analyseur lexical, que l'on surnommera Gal pour faire plus simple, similaire au logiciel Flex mais entièrement programmé en C par mes soins. A partir d'une expression rationnelle « e » dite simple (c'est-à-dire définie uniquement à l'aide de la concaténation, l'union et l'étoile de Kleene) le Gal doit construire :

- L'arbre de syntaxe abstraite (ast) de e
- L'automate fini non déterministe (nfa) associé à l'ast
- L'automate fini déterministe (dfa) associé au nfa
- L'automate fini déterministe minimal

Et à partir de ce dernier le programme devra engendrer le code d'un programme qui permettra de savoir si un mot appartient ou non au langage rationnel de e.

Aussi, le programme devra fournir une représentation graphique de chacun des objets cités plus haut.

Structure du projet :

Mon projet se constituera de ces fichiers :



Chacune des étapes citées plus haut correspond à un couple de fichiers .h et .c (l'utilité et l'implémentation de chacun des fichiers sera détaillée plus tard), le fichier .h servant de « plan » pour la réelle implémentation dans le fichier .c. Le fichier Makefile permettra de compiler l'ensemble des fichiers du Gal. Le fichier main.c rassemblera tout ce qui a été écrit dans les autres fichiers pour remplir toutes les tâches citées plus haut.

Ast :

Ceci constitue la première étape de notre implémentation. Lorsque le Gal reçoit une expression rationnelle il doit d'abord savoir comment le lire et quelles sont les priorités de

l'expression afin de pouvoir construire son arbre de syntaxe abstraite, cette partie comportera donc les bases de construction de l'arbre de syntaxe abstraite bien que l'assemblage de celui-ci soit relégué au code lecteur_exp.

Ce code comporte plusieurs fonctions :

- 4 commençant par les mots « créer nœud » qui comme leur nom l'indique vont créer le nœud associé à chaque élément de l'expression donnée selon sa nature,
- puis une fonction graphe_ast qui à l'aide d'une fonction écrire_arbre définie uniquement dans ast.c va générer une représentation de l'arbre de syntaxe abstraite.

Lecteur_exp :

Cette partie décrit la lecture d'une expression régulière par le Gal. Ici, la difficulté était de respecter les priorités de lecture d'une expression régulière (union<concatenation<étoile de Kleene<caractère simple) j'ai fait le choix de créer une unique fonction commune à l'ensemble du projet lire_expression qui va utiliser de manière directe ou indirecte des autres fonctions propres au fichier lecteur_exp.c. Ce dernier comporte 4 fonctions analyse_... qui vont chacune appeler la fonction de priorité supérieure (sauf analyse_element qui est de priorité la plus haute) afin de descendre petit à petit vers les feuilles de l'arbre syntaxique. Chaque fonction analyse_... crée également le nœud associé à l'élément lu. Ainsi, la fonction lire_expression s'occupera, comme son nom l'indique, de lire l'expression mais également de créer l'arbre de syntaxe abstraite global de l'expression en respectant les priorités de cette dernière.

Nfa :

On va passer à la création du nfa, au lieu de le construire à partir de l'expression régulière (je pense que ça aurait été possible mais très compliqué) je le construis à partir de l'ast à l'aide de l'algorithme de McNaughton, Yamada et Thompson. La complexité de ceci a été de savoir quelles fonctions implémenter pour être efficace sans me répéter inutilement. J'ai donc commencé par la « base », la fonction de création d'un état, il est associé à un symbole, un numéro et a aux plus deux états sortants comme l'algorithme indique qu'un état a aux plus deux transitions. Ensuite, comme l'unité de base dans un nfa est le nfa associé à un caractère unique ensuite pour chaque action on associe des « petits nfa » entre eux ; c'est ce que j'ai défini dans les fonctions ..._nfa. Enfin, la fonction ast_to_nfa parcourt l'ast et crée les états et transitions associées. Comme pour l'ast on construit ensuite le graphe du nfa à l'aide d'une fonction dédiée.

Dfa :

L'algorithme de passage du Nfa au Dfa utilise une table de transition : on rassemble les états qui correspondent au même mot lu et les états vers lesquels ils transitent par quelle transition jusqu'à ce qu'on ait des nouveaux états qui peuvent lire exactement les mêmes mots que le Nfa. J'avais pensé à écrire d'abord une fonction pour créer un ensemble d'états représentant le même mot mais ne savais pas comment coder ça d'une manière où le programme pourrait

comprendre quels états exactement sont dans l'ensemble. J'ai alors trouvé une solution : pour chaque ensemble d'états au lieu d'écrire l'ensemble sous la forme $\{q_1, q_2, q_5\}$ je les écris sous la forme $\{1, 1, 0, 0, 1\}$, les 1 correspondent au fait que l'état aux indices où ils se trouvent appartient à l'ensemble d'état. J'ai d'abord écrit les deux fonctions décrites dans le cours :

`epsilon_cloture`(pour trouver les états reliés par des `epsilon_transition`) et `transition`(qui servira dans la fonction finale à passer aux prochains états par un caractère). Ensuite, j'ai défini trois fonctions qui seront utiles au bon fonctionnement de la fonction `nfa_to_dfa`. Cette dernière rassemble toutes les fonctions qui la précède et effectue la réelle conversion. Elle commence par initialiser le premier état du dfa en lui ajoutant l'état initial du nfa et les états qu'on peut atteindre depuis ce dernier par les epsilon transitions, on parcourt ensuite la liste des nouveaux états du dfa créés et pour chaque caractère de l'alphabet on vérifie si l'état sur lequel on est possède une transition soit avec un caractère soit une epsilon transition et on crée un nouvel état si celui qu'on a atteint n'avait pas encore été créé. A la fin on marque les états contenant l'état final du nfa comme états finaux.

Minimisation_dfa :

Le principe de la minimisation est de créer un nouveau dfa avec moins d'états possible qui reconnait le même langage que le dfa initial. On utilise l'algorithme décrit dans le cours pour définir la fonction `dfa_minimal`. On va alors regrouper les états qui n'ont pas de chaîne qui les distingue. Ces états regroupés vont alors devenir un nouvel état qui aura les mêmes transitions. On aura alors une partition de groupes d'états et chaque ensemble de cette partition sera un état du dfa minimal. Pour ce faire, j'ai d'abord défini 3 fonctions qui seront utiles à la minimisation : une qui vérifie vers quel état on va en lisant une certaine lettre, une autre qui vérifie dans quelle direction chaque état va et une dernière qui créera les états pour le dfa min final. La fonction `dfa_minimal` va d'abord séparer les états du dfa en deux groupes le ou les état/s final/aux du dfa et les autres états on vérifie alors si les états d'un même groupe ont la même direction. Dans ce cas là ils restent dans le même groupe autrement on sépare les états qui diffèrent du groupe et on continue ainsi jusqu'à ce que le nombre de groupes soit stable. Enfin, on fusionne chaque groupe pour en faire un état unique du dfa minimal.

La fonction de représentation graphique du dfa minimal est identique à celle du dfa car les deux sont des dfa.

Generateur :

Ceci est la toute dernière étape de notre Gal, le programme de ce fichier devra à partir de l'automate minimal engendrer le code d'un programme C qui implantera un analyseur lexical qui pourra déterminer si un mot appartient ou non au langage défini par l'expression rationnelle donnée. La difficulté première de cette partie a été de comprendre comment générer le code et comment faire qu'il s'écrive et s'implémente bien dans tous les cas de figure. La méthode utilisée est similaire à celle de représentation graphique mais au lieu de créer un fichier .dot on crée un fichier de code .c dans lequel on imprime à l'aide de `fprintf` le code de l'analyseur lexical. Il a fallu bien faire attention aux indentations, aux sauts de lignes et à la logique globale du code. Pour cela j'ai partiellement écrit le code de l'analyseur lexical dans un

fichier à part et l'ai copié coller avec les indentations quand c'était possible. Au début du code le programme va demander à l'utilisateur de saisir un mot à tester, puis le programme en partant de l'état initial de l'automate va essayer de parcourir l'automate à l'aide de la commande pour former le mot reçu. S'il n'existe pas dans l'automate (transitions manquantes) ou ne se finit pas sur un état final le programme s'arrête et affiche le message indiquant que le mot n'appartient pas au langage, et s'il appartient bien au langage le code affichera le message adapté.

Main :

C'est la finalisation de tout ce projet, on extrait d'abord la première ligne du fichier donné contenant l'expression régulière et on utilise toutes les fonctions principales de ce qui a été cité plus haut pour successivement créer l'ast, le nfa, le dfa et le dfa minimal associés à l'expression régulière. On crée également au fur et à mesure les graphes associés. Enfin, on génère le code C qui implémentera l'analyseur lexical.